

Resumen Teorico – Practico 4

Testing basado en grafos

Cobertura estructural y CFG

Testing de Software 2024

Índice

Introduccion	2
1. Grafos como estructura de modelado	2
1.1. Por que grafos	2
1.2. Definicion formal de grafo	3
1.3. Caminos, longitud y subcaminos	3
1.4. Alcanzabilidad	3
1.5. Grafos SESE	4
1.6. Visita, recorrido y notacion	4
2. Testing y cobertura de grafos	4
2.1. Esquema general	4
2.2. Tipos de criterios sobre grafos	5
3. Criterios estructurales	5
3.1. Cobertura de nodos (NC)	5
3.2. Cobertura de aristas (EC)	5
3.3. Cobertura de pares de aristas (EPC)	6
3.4. Complete Path Coverage (CPC)	6
3.5. Caminos simples y caminos principales (prime paths)	6
3.6. Prime Path Coverage (PPC)	6
3.7. Computo de prime paths: tabla por longitud	7
4. Tours, sidetrips y detours	7

5. Requisitos infactibles y Best Effort Touring	8
5.1. Requisitos infactibles	8
5.2. Best Effort Touring	9
6. Relacion entre criterios: subsumption	9
7. Grafos de Control de Flujo (CFG)	10
7.1. Definicion	10
7.2. Construcccion del CFG a partir del codigo	10
7.3. Criterios sobre CFG	11
7.4. Ejemplo: computeStats	11
8. Otros grafos utiles en testing de codigo	12
8.1. Grafo de llamadas (call graph)	12
8.2. Otros modelos	12
9. Glosario rapido	12
Cierre	13

Introduccion

Este resumen sintetiza el material teorico asociado al TP4:

- *Introduction to Software Testing* (Ammann & Offutt, 2da edicion), Capitulo 7: **Graph Coverage**.
- *Notas 07: Criterios de cobertura de grafos – Introduccion* (V. Bengolea).
- *Notas 08: Criterios de cobertura de grafos desde codigo fuente* (V. Bengolea).

El objetivo del resumen es servir de referencia para repasar: definiciones formales, criterios estructurales sobre grafos (NC, EC, EPC, CPC, PPC), conceptos de *tour*, *sidetrip*, *detour*, *infeasibility*, *Best Effort Touring*, y aplicaciones al codigo fuente mediante el grafo de control de flujo (CFG).

1. Grafos como estructura de modelado

1.1. Por que grafos

Los grafos son la *estructura mas utilizada* para testing. Permiten modelar:

- flujo de control de un programa (CFG),
- modelos de diseno (UML, secuencia, actividad),
- maquinas de estados finitos (FSM) y *statecharts*,
- casos de uso, flujo de datos, llamadas entre metodos, etc.

Idea operativa. Una vez que el software esta modelado como un grafo, el testing se convierte en *recorrer* ese grafo siguiendo reglas explicitas: las reglas son los criterios de cobertura.

Idea clave

“Definir un modelo del software y encontrar formas de recorrerlo” es la receta basica del diseno de tests guiado por modelos (MDTD).

1.2. Definicion formal de grafo

Definicion

Un **grafo** consta de:

- un conjunto *no vacio* de nodos N ;
- un conjunto *no vacio* de nodos iniciales $N_0 \subseteq N$;
- un conjunto *no vacio* de nodos finales $N_f \subseteq N$;
- un conjunto de aristas $E \subseteq N \times N$. Cada arista (n_i, n_j) tiene *predecesor* n_i y *sucesor* n_j .

Convencion grafica: el nodo final se dibuja con doble circulo; el nodo inicial recibe una flecha “de la nada”.

Cuidado. Si $N_0 = \emptyset$, el grafo no es valido para testing: no hay punto desde donde empezar la ejecucion. Lo mismo si $N_f = \emptyset$: no hay donde terminar.

1.3. Caminos, longitud y subcaminos

Definicion

Camino. Secuencia $[n_1, n_2, \dots, n_M]$ tal que cada par consecutivo (n_i, n_{i+1}) esta conectado por una arista.

Longitud. Numero de aristas (es decir, $M - 1$).

Subcamino. Una subsecuencia consecutiva de un camino p es un subcamino de p . (Cada arista, mirada como un camino de longitud 1, es tambien un subcamino.)

Camino de test. Camino que empieza en algun nodo inicial y termina en algun nodo final. *Es la abstraccion de una ejecucion de test.*

Idea clave

Software determinista: cada test ejecuta exactamente un camino de test (relacion many-to-one: muchos tests pueden ejecutar el mismo camino).

Software no determinista: el mismo test puede ejecutar distintos caminos en distintas corridas.

1.4. Alcanzabilidad

Definicion

Un nodo n es **sintacticamente alcanzable** desde n_i si existe un camino de n_i a n .

Un nodo n es **semanticamente alcanzable** si existe *algun* input que efectivamente hace que la ejecucion lo visite (es decir, el camino sintactico es factible).

Los criterios estructurales razonan sobre alcanzabilidad sintactica; la semantica entra cuando se discute si un requisito de test es *factible* o *infactible* (infeasible).

1.5. Grafos SESE

Definicion

Un grafo es **SESE** (*Single Entry, Single Exit*) si:

- $|N_0| = 1$ y $|N_f| = 1$ (un unico nodo inicial n_0 y un unico nodo final n_f);
- n_f es sintacticamente alcanzable desde todos los nodos de N ;
- ningun nodo es sintacticamente alcanzable desde n_f , salvo que $n_f = n_0$.

El CFG de un metodo se modela tipicamente como SESE: un punto de entrada (el **return**, o un nodo virtual al final) y un unico punto de salida.

1.6. Visita, recorrido y notacion

- Un camino p **visita** un nodo n si n esta en p . Analogamente para aristas.
- Un camino p **recorre** (tours) un subcamino q si q es subcamino de p .
- $\text{camino}(t)$: el camino de test que el test t ejecuta.
- $\text{caminos}(T)$: el conjunto de caminos ejecutados por la suite T .

2. Testing y cobertura de grafos

2.1. Esquema general

1. Desarrollar un modelo del software como un grafo G .
2. Definir un **criterio de cobertura** C sobre G . El criterio genera un conjunto de **requisitos de test** TR (cada uno describe una propiedad que los caminos de test deben satisfacer).
3. Encontrar un conjunto de tests T tal que para todo $tr \in TR$ exista al menos un camino en $\text{caminos}(T)$ que recorra/visite tr .

Definicion

Satisfaccion. T satisface C sobre G sii $\forall tr \in TR, \exists p \in \text{caminos}(T) :$
 p recorre/visita tr .

2.2. Tipos de criterios sobre grafos

- **Estructurales:** definidos sobre nodos y aristas del grafo. Son los que cubre el practico.
- **Sobre flujo de datos:** requieren un grafo *anotado* con definiciones y usos de variables. (Fuera del alcance de esta asignatura.)

3. Criterios estructurales

3.1. Cobertura de nodos (NC)

Definicion

Node Coverage (NC). T satisface NC sobre G sii para cada nodo sintacticamente alcanzable n , existe $p \in \text{caminos}(T)$ que visita n .
Equivalentemente, TR contiene todos los nodos alcanzables de G .

Ejemplo

En el grafo $1 \rightarrow \{2, 3\} \rightarrow 4 \rightarrow \{5, 6\} \rightarrow 7$, los caminos $p_1 = [1, 2, 4, 5, 7]$ y $p_2 = [1, 3, 4, 6, 7]$ visitan todos los nodos: $\{1, 2, 3, 4, 5, 6, 7\}$. Cualquier suite T con $\text{camino}(t_1) = p_1$ y $\text{camino}(t_2) = p_2$ satisface NC.

3.2. Cobertura de aristas (EC)

Definicion

Edge Coverage (EC). TR contiene cada camino de longitud *a lo sumo* 1 incluido en G .

“A lo sumo 1” significa que tambien se incluyen los caminos de longitud 0 (nodos aislados), de manera que EC pueda definirse incluso en grafos sin aristas. Esto permite que **EC subsuma a NC**: si una suite cubre todas las aristas, automaticamente visita todos los nodos.

Idea clave

NC y EC *difieren* solo cuando hay dos nodos unidos por una arista y por otro subcamino. El caso canonico es un **if** sin **else**: el cuerpo del **if** crea un nodo extra, pero la arista que lo evita es propia de EC y no de NC.

Mapeo a herramientas reales.

- Cobertura de nodos = *Statement Coverage* (cobertura de sentencias).

- Cobertura de aristas = *Branch Coverage* (cobertura de ramas).

3.3. Cobertura de pares de aristas (EPC)

Definicion

Edge-Pair Coverage (EPC). TR contiene cada camino de longitud *a lo sumo* 2 incluido en G .

EPC obliga a cubrir cada secuencia de *dos* aristas consecutivas. Por la clausula “a lo sumo 2”, sigue siendo aplicable a grafos con menos de dos aristas. EPC subsume EC, que subsume NC.

Ejemplo

Para un grafo con $\{(1, 4), (2, 4), (3, 4), (4, 5), (4, 6)\}$ los pares de aristas son $[1, 4, 5]$, $[1, 4, 6]$, $[2, 4, 5]$, $[2, 4, 6]$, $[3, 4, 5]$, $[3, 4, 6]$, mas las aristas mismas como caminos de longitud 1.

3.4. Complete Path Coverage (CPC)

Definicion

Complete Path Coverage (CPC). TR contiene *todos* los caminos de G .

CPC es el criterio mas exigente. Si el grafo tiene un bucle (cycle), la cantidad de caminos es **infinita** y CPC *no se puede realizar*. De aqui nace la necesidad de un sustituto mas razonable.

3.5. Caminos simples y caminos principales (prime paths)

Definicion

Camino simple. Camino donde no se repiten nodos, *salvo posiblemente* el primero y el ultimo (es decir, una vuelta de bucle es simple, pero un camino con bucles *internos* no lo es).

Camino principal (*prime path*). Camino simple que no es subcamino propio de ningun otro camino simple. Equivalentemente: un camino simple *maximal*.

3.6. Prime Path Coverage (PPC)

Definicion

Prime Path Coverage (PPC). TR contiene todos los caminos principales de G .

Propiedades.

- PPC es *finito* (a diferencia de CPC): todo grafo tiene un numero finito de caminos simples y por lo tanto de prime paths.
- PPC requiere que los bucles se ejecuten *y* se salten (cero veces, una vez, mas de una vez).
- PPC subsume NC y EC.
- PPC *casi* subsume EPC, pero no siempre: si un nodo n tiene un bucle hacia si mismo (arista (n, n)), EPC exige $[n, n, m]$ y $[m, n, n]$, pero ninguno de los dos es un camino simple ni un subcamino de uno. Por lo tanto, no es prime path.

Ejemplo

Si el grafo es $1 \rightarrow 2 \rightarrow 3$ con un bucle $2 \rightarrow 2$:

- $TR_{EPC} = \{[1, 2, 3], [1, 2, 2], [2, 2, 3], [2, 2, 2]\}$.
- $TR_{PPC} = \{[1, 2, 3], [2, 2]\}$.

PPC pide menos requisitos, pero EPC tiene requisitos que no son subcaminos de ningun prime path.

3.7. Computo de prime paths: tabla por longitud

Una tecnica clasica para enumerar prime paths:

1. Listar todos los caminos simples de longitud 0 (los nodos individuales).
2. Extender a longitud 1, 2, 3, ... agregando una arista por vez sin repetir nodos.
3. Marcar los caminos que *no se pueden extender* (terminaron en un nodo final o en uno sin sucesores nuevos) y los caminos *ciclicos* (vuelven al inicio).
4. Eliminar los caminos que son subcamino propio de alguno mas largo. Lo que queda son los prime paths.

4. Tours, sidetrips y detours

Los caminos primarios no tienen bucles internos, pero los caminos de *test* si pueden tenerlos. La diferencia entre lo que un test pide y lo que el codigo permite hace necesarias tres formas distintas de “visitar” un subcamino.

Definicion

Sea p un camino de test y q un subcamino objetivo.

- **Tour.** p recorre q si q es subcamino directo de p .
- **Tour con sidetrip** (viaje adicional). p recorre q con sidetrips sii cada *arista* de q esta en p en el mismo orden, aunque entre medias haya desviaciones que vuelvan al mismo nodo del que salieron.
- **Tour con detour** (desvio). p recorre q con detours sii cada *nodo* de q esta en p en el mismo orden, aunque la conexion entre dos nodos consecutivos de q se haga por otra ruta (que reentra por un sucesor del nodo de salida, no por el mismo nodo).

Diferencia practica. Sidetrip exige que el desvio vuelva exactamente al mismo nodo; detour solo exige que vuelva a algun sucesor del camino. Por eso detour es mas permisivo.

Ejemplo

Sobre $q = [1, 2, 3, 5, 6]$:

- Tour directo: $[1, 2, 3, 5, 6]$.
- Tour con sidetrip: $[1, 2, 3, 4, 3, 5, 6]$ (desvio $3 \rightarrow 4 \rightarrow 3$ que vuelve a 3).
- Tour con detour: $[1, 2, 3, 4, 5, 6]$ (en lugar de $3 \rightarrow 5$, se va $3 \rightarrow 4 \rightarrow 5$).

5. Requisitos infactibles y Best Effort Touring

5.1. Requisitos infactibles

Definicion

Un requisito de test es **infactible** si no existe ningun input que lo haga ejecutar.

Causas tipicas:

- Codigo muerto.
- Subcamino que solo se ejecutaria con un input contradictorio (por ejemplo, $x > 0 \wedge x < 0$).
- Caminos que requieren bucles que el codigo siempre ejecuta al menos una vez (cuando el cuerpo no puede saltarse).

Mala noticia. Decidir si un requisito es factible es *indecidable* en general. Por lo tanto, casi nunca se alcanza el 100 % de cobertura en la practica.

5.2. Best Effort Touring

Idea clave

Best Effort Touring (BET). Para cada requisito de test:

- satisfacerlo *directamente* (tour sin sidetrips) si es posible;
- si no, permitir un tour con sidetrips;
- si ni siquiera así es factible, justificarlo como infactible y declararlo no cubierto.

BET es un compromiso entre dos extremos:

- **Sin sidetrips:** muchos requisitos quedan infactibles aunque su intención se haya cumplido por otro camino.
- **Siempre con sidetrips:** el criterio se debilita, porque cualquier camino que pase “cerca” del requisito lo da por cubierto.

BET ofrece la máxima rigurosidad posible sin descartar requisitos solo porque no se puedan recorrer *contiguamente*.

6. Relacion entre criterios: subsumption

Definicion

C_1 *subsume* a C_2 sii *todo* conjunto de tests que satisface C_1 también satisface C_2 .

Jerarquia para los criterios estructurales:

$$CPC \rightarrow PPC \rightarrow EPC \rightarrow EC \rightarrow NC,$$

donde $C_1 \rightarrow C_2$ significa “ C_1 subsume C_2 ”. La única salvedad es el bucle propio $((n, n))$, donde PPC no subsume EPC.

Consecuencia practica. Para cubrir un criterio basta cubrir cualquiera que lo subsuma. PPC es el criterio más fuerte que se puede satisfacer en finito tiempo en grafos con bucles.

7. Grafos de Control de Flujo (CFG)

7.1. Definicion

Definicion

Un **Grafo de Control de Flujo (CFG)** modela todas las ejecuciones posibles de un programa mediante su estructura de control.

- **Nodos:** instrucciones o *bloques basicos* (secuencias de instrucciones tales que, si se ejecuta la primera, se ejecutan todas las demas, sin ramificacion intermedia).
- **Aristas:** transferencias de control.
- Opcionalmente se anotan con *predicados* en las ramas, definiciones y usos de variables.

Bloque basico. Es la unidad natural de modelado del CFG: agrupa sentencias secuenciales que “van juntas” desde el punto de vista del control.

7.2. Construccion del CFG a partir del codigo

- **Secuencia.** Las sentencias del mismo bloque basico se colapsan en un solo nodo.
- **if-else.** Un nodo de condicion con dos aristas salientes (true/false); los dos cuerpos terminan en un nodo de merge.
- **if sin else.** Un nodo de condicion, una rama que ejecuta el cuerpo, otra que lo evita; ambas terminan en el sucesor.
- **if-return.** La rama true va al **return** y *no* continua al resto del flujo; no hay arista entre el bloque de **return** y el sucesor.
- **while.** Un nodo de condicion con dos aristas: hacia el cuerpo (true) y hacia la salida (false). El cuerpo vuelve al nodo de condicion.
- **for.** Equivalente a un **while** con un nodo extra de inicializacion y otro de actualizacion (que se ejecuta antes de volver a la condicion).
- **do-while.** El cuerpo se ejecuta primero; la condicion al final decide si se vuelve.
- **break / continue.** Aristas extra desde el bloque que las contiene: **break** salta al sucesor del bucle; **continue** vuelve a la condicion (o al paso de actualizacion en un **for**).
- **switch.** Un nodo de condicion con k aristas salientes (una por **case** y otra por **default**). Si un **case** no tiene **break**, hay una arista que “cae” al siguiente **case** (*fallthrough*).
- **try-catch.** Cada sentencia que puede lanzar una excepcion tiene una arista hacia el

bloque `catch`; las excepciones propias del bloque `try` pueden modelarse explícitamente con un nodo `throw`.

7.3. Criterios sobre CFG

Maapeo directo entre los criterios genericos y la nomenclatura habitual:

- Cobertura de nodos = cobertura de sentencias (*statement coverage*).
- Cobertura de aristas = cobertura de ramas (*branch coverage*).

EPC y PPC se aplican igual que en grafos abstractos: cada par de aristas (resp. cada prime path) del CFG es un requisito de test.

7.4. Ejemplo: `computeStats`

Un metodo con dos bucles `for` consecutivos, asignaciones intermedias y bloques de impresion final. Un CFG tipico tendria 8 nodos:

1. Inicializacion (`length, sum = 0`).
2. `i = 0` del primer `for`.
3. Condicion del primer `for`.
4. Cuerpo del primer `for` (`sum += numbers[i], i++`).
5. Bloque intermedio (`med, mean, varsum = 0, i = 0`).
6. Condicion del segundo `for`.
7. Cuerpo del segundo `for` (`varsum += ..., i++`).
8. Bloque final (varios `System.out.println`).

con aristas que cierran cada bucle ($3 \rightarrow 4 \rightarrow 3$ y $6 \rightarrow 7 \rightarrow 6$) y aristas de salida ($3 \rightarrow 5$, $6 \rightarrow 8$). Sobre ese CFG se calculan los TR para EC, EPC y PPC del mismo modo que en los grafos abstractos.

8. Otros grafos utiles en testing de codigo

8.1. Grafo de llamadas (call graph)

Definicion

Grafo de llamadas. Nodos = unidades (en Java, metodos). Aristas = llamadas entre unidades.

Es el tipo de grafo mas comun para *testing estructural a nivel de disenio*.

- Cobertura de nodos = **cobertura de metodos** (llamar a cada metodo al menos una vez).
- Cobertura de aristas = **cobertura de llamadas** (ejecutar cada llamada al menos una vez).

En programacion orientada a objetos el enfasis en modularidad y reuso desplaza la complejidad a las *conexiones* entre componentes, por lo que testear esas conexiones se vuelve particularmente importante.

8.2. Otros modelos

Los criterios estructurales sobre grafos tambien se aplican a:

- Grafos de flujo de datos (DD-graphs), con anotaciones de definiciones y usos de variables.
- Maquinas de estados finitos (FSM) y *statecharts*.
- Grafos de casos de uso o de actividad UML.
- Grafos de integracion entre modulos.

9. Glosario rapido

- **CFG:** *Control Flow Graph* – grafo de flujo de control.
- **NC:** *Node Coverage* (cobertura de nodos / sentencias).
- **EC:** *Edge Coverage* (cobertura de aristas / ramas).
- **EPC:** *Edge-Pair Coverage* (cobertura de pares de aristas).
- **CPC:** *Complete Path Coverage* (cobertura de todos los caminos).
- **PPC:** *Prime Path Coverage* (cobertura de caminos principales).
- **SESE:** *Single Entry, Single Exit*.

- **Camino simple:** sin nodos repetidos, salvo opcionalmente el primero y el ultimo.
- **Camino principal:** camino simple maximal.
- **Tour:** subcamino recorrido contiguamente.
- **Sidetrip:** desvio que vuelve al mismo nodo.
- **Detour:** desvio que vuelve a un sucesor del nodo.
- **Infeasible TR:** requisito de test que ningun input puede satisfacer.
- **Best Effort Touring:** tour directo donde se pueda, sidetrips para el resto.

Cierre

Los grafos son una abstraccion muy potente para disenar tests: capturan la estructura de control del programa, se traducen mecanicamente a requisitos de test, y permiten elegir el nivel de exigencia (NC, EC, EPC, PPC) segun el presupuesto. La nocion de tour, sidetrip y detour, junto con *Best Effort Touring*, ofrecen el lenguaje necesario para manejar los requisitos infactibles sin renunciar a la rigurosidad del criterio elegido.